

Procesamiento de Transacciones Distribuidas

X/Open XA · commit en dos fases (2PC) · variantes en árbol y D2PC · commit en tres fases (3PC)

Bases de Datos (Relacionales) · Transacciones, Concurrency y Recuperación

Ing. Informática · Lic. en Cs. de Datos

Ricardo Di Pasquale

Alejandro Isidro

Edgardo Sanchez

Delfina Las Heras

AGENDA

1. Qué es una transacción distribuida
2. El modelo X/Open XA: componentes
3. Roles e interfaces (TX, XA, XA+...)
4. Commit en dos fases (2PC)
5. Transacciones "en duda" y bloqueo
6. Variantes: 2PC en árbol y D2PC
7. Commit en tres fases (3PC)

DEL CENTRALIZADO AL DISTRIBUIDO

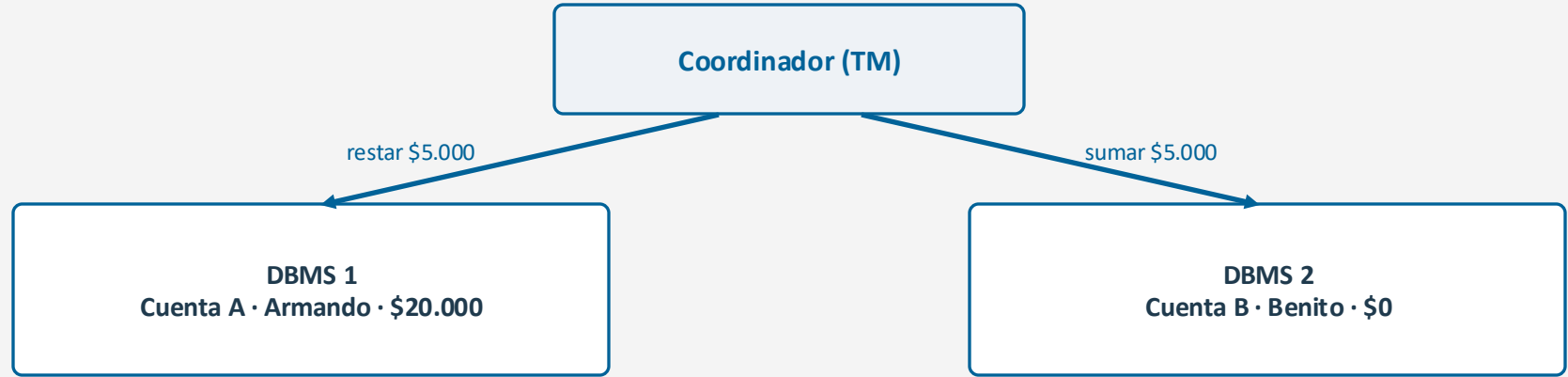
¿Qué es una transacción distribuida?

Coordinar transacciones entre varias bases

Cuando administrar los datos de forma centralizada es imposible o inconveniente, aparecen las bases distribuidas. Un **Coordinador de Transacciones Distribuidas** orquesta operaciones sobre varias instalaciones como si fueran una sola base, garantizando ACID global.

- ▶ **Es una forma de OLTP:** una transacción global combina operaciones de varias aplicaciones sobre una o más bases, en uno o más equipos vía red.
- ▶ **Alcance del tema:** hoy nos centramos en el manejo de transacciones; quedan afuera replicación, confiabilidad y consultas distribuidas.

El caso: transferencia entre dos bases



Si DBMS 2 cae en el paso final: $A = \$15.000$ y $B = \$0 \rightarrow$ se pierden \$5.000. La transacción global debe ser atómica (todo o nada) en ambas bases.

EL ESTÁNDAR DE LA INDUSTRIA

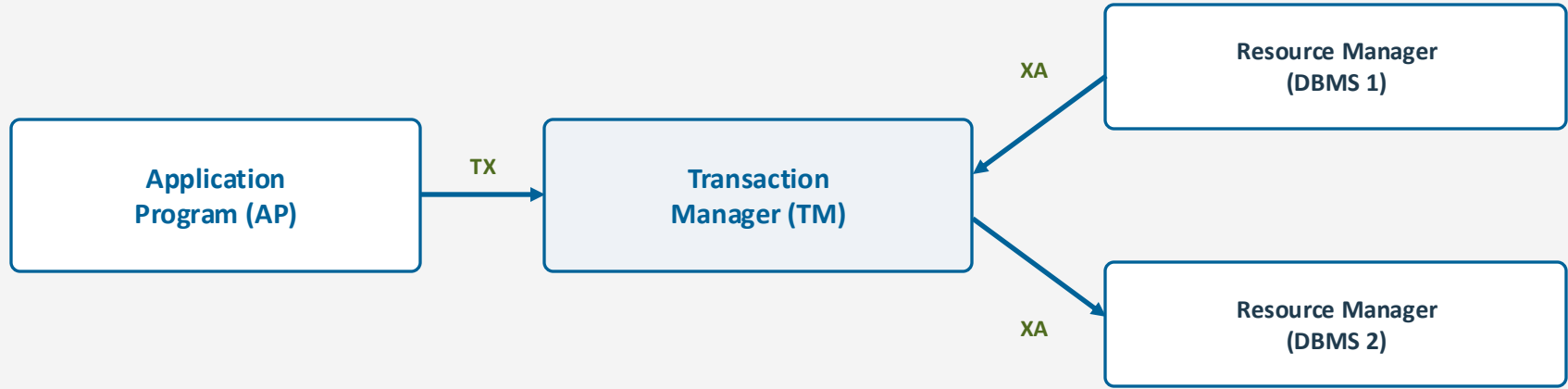
El modelo X/Open XA

Componentes del modelo X/Open XA

Componente	Rol
Application Program (AP)	la lógica de negocio (una o varias aplicaciones)
Transaction Manager (TM)	coordina las transacciones globales y las sincroniza
Resource Manager (RM)	administra recursos: DBMS, colas (JMS), middleware
Communications RM (CRM)	RM especializado para comunicar datos entre aplicaciones

Estándar abierto del Open Group. Lo adoptaron tecnologías como EJB (Java) y MTS (Microsoft).

Arquitectura XA: quién habla con quién



El AP accede a cada RM vía SQL (AP-RM). El CRM (interfaces CM / XA+) comunica aplicaciones entre equipos.

Las interfaces de la API DTP

Interfaz	Entre	Para qué
AP-RM (SQL)	aplicación ↔ RM	actualizar un recurso (p. ej. SQL)
TX	aplicación ↔ TM	arrancar transacciones globales
CM	aplicación ↔ CRM	comunicarse con otras apps (cliente/servidor, P2P, RPC)
XA	TM ↔ RM	coordinar la transacción global → habilita el 2PC
XA+	TM ↔ CRM	coordinar transacciones entre computadoras (DTP)
System	componentes internos	interacciones del sistema (invisibles al programador)

PONERSE DE ACUERDO

Commit en dos fases (2PC)

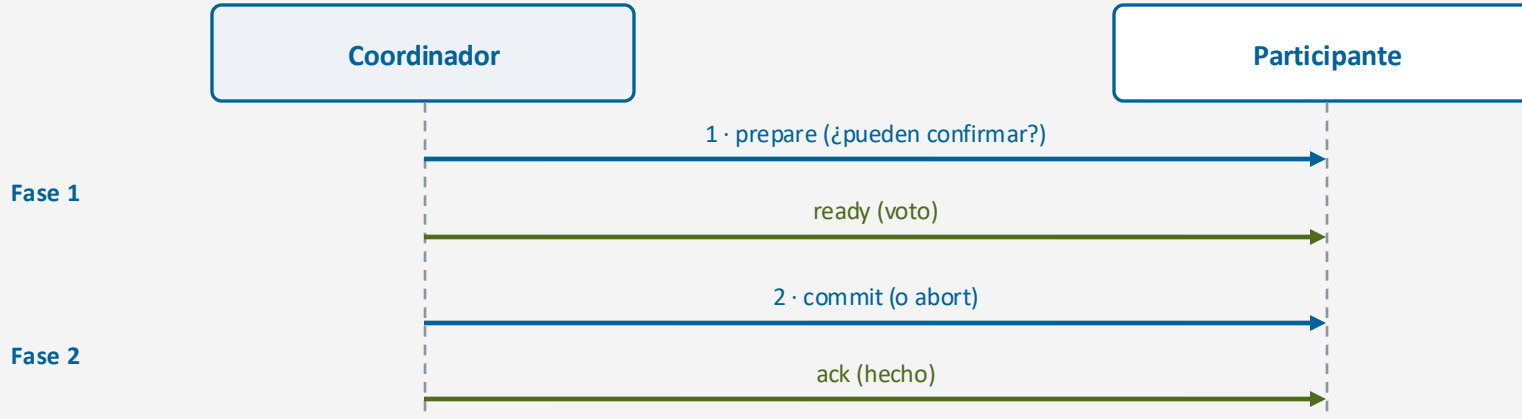
Commit en dos fases (2PC)

Permite que todos los nodos de la transacción global se pongan de acuerdo para confirmar, garantizando ACID. El TM que inicia es el **coordinador**; las bases involucradas son los **participantes**.

- ▶ **Fase 1 — Petición (prepare):** el coordinador pregunta a cada participante si puede confirmar; todos votan.
- ▶ **Fase 2 — Commit:** si todos votaron sí → commit a todos; si alguno no → abort a todos.

El coordinador asigna un GTRID (id global de la transacción) y etiqueta con él todos los mensajes para que cada RM los rastree.

2PC · secuencia de mensajes



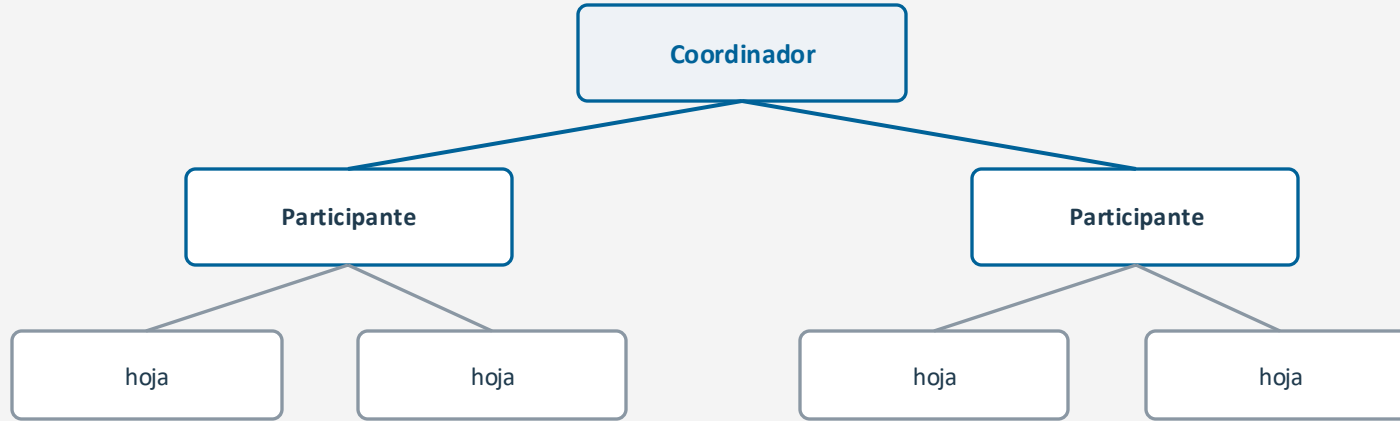
Se muestra un participante; participan todos. El coordinador registra el estado global en su TLog.

Dos problemas del 2PC

Transacciones "en duda" (in-doubt): si en la fase 2 el ack no llega (falla de comunicación), el participante no sabe si el commit se hizo o no. El coordinador, tras un timeout, se predispone a abortar.

2PC es bloqueante: un participante que votó "ready" queda bloqueado esperando la decisión. Si el coordinador cae de forma permanente, ese participante no resuelve su transacción y mantiene los recursos comprometidos, frenando a todos los que compiten por esos locks.

Variantes: 2PC en árbol y D2PC



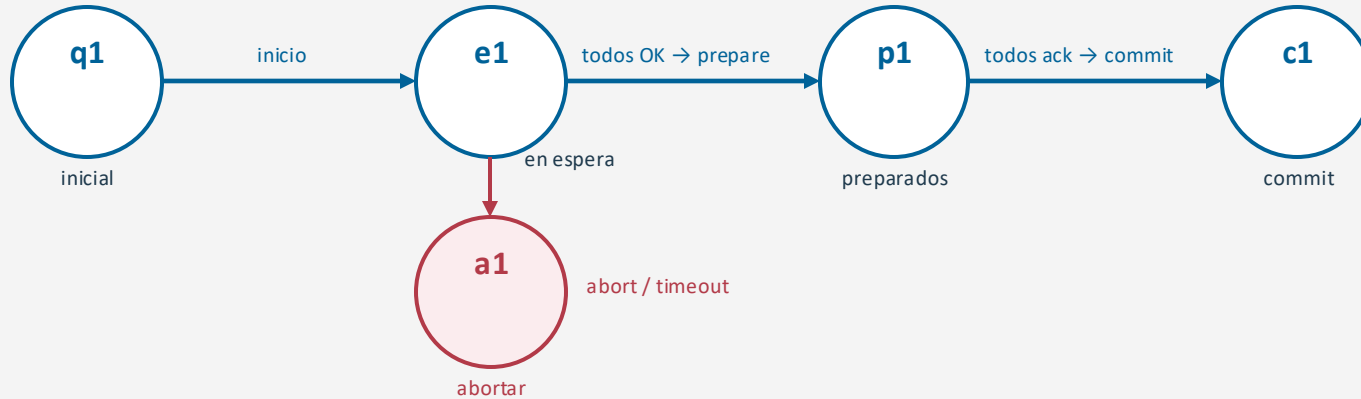
En árbol: los mensajes bajan y los ack suben recolectándose ("abortar" sube de inmediato). D2PC: sin coordinador fijo — se elige dinámicamente donde se concentran los ack; libera recursos antes.

EVITAR EL BLOQUEO

Commit en tres fases (3PC)

3PC · máquina de estados (coordinador)

Agrega una fase intermedia ("preparados") y una **cota superior de tiempo** antes de commit/abort: así un nodo puede liberar sus locks tras el timeout en vez de bloquearse.



2PC vs. 3PC

Aspecto	2PC	3PC
Fases	2 (prepare, commit)	3 (prepare, preparados, commit)
¿Bloqueante?	Sí, si cae el coordinador	No: cota superior de tiempo (timeouts)
Ante caída de un nodo	recursos comprometidos / in-doubt	libera los locks al vencer el timeout
Debilidad principal	el bloqueo	no se recupera si la red se particiona

Talón de Aquiles del 3PC: si la red se segmenta (partición), el protocolo no puede recuperarse.

Síntesis

- ▶ Una transacción distribuida abarca varias bases y debe ser ACID en todas.
- ▶ X/Open XA estandariza AP, TM, RM y CRM, y las interfaces (TX, XA, XA+...).
- ▶ El coordinador (TM) y los participantes (RM) acuerdan con 2PC: prepare + commit.
- ▶ 2PC es simple pero bloqueante: aparecen transacciones "en duda".
- ▶ Variantes en árbol y D2PC mejoran la propagación y liberan recursos antes.
- ▶ 3PC evita el bloqueo con una fase extra y timeouts, pero falla ante particiones de red.

¡Muchas gracias! Q&A

Seguinos



@ingeagrariasuca



@IngeagrariasUCA



Ingeniería y Ciencias Agrarias

Contacto



Av. Alicia Moreau de Justo 1500
C1107AAZ, Buenos Aires, Argentina



0810-220-0822



inge_agrarias@uca.edu.ar